

TPI FoodStore - Primera entrega parcial

Informe técnico de unidades 1, 2 y 3

Grupo: Franco Sampieri, Mauro Di Pietro, Renzo Victoria, Tomas Martinez, Alam Gonzalez

Repositorio del proyecto: https://github.com/francosampieri/TP2_BDD2_FoodStore.git

1. Alcance del informe

FoodStore es un sistema de gestión de pedidos gastronómicos implementado íntegramente en PostgreSQL 17. Este informe resume la evidencia de integridad, transacciones, concurrencia, optimización, índices, vistas y objetos programables desarrollada hasta la primera entrega parcial del TPI.

Criterio	Evidencia principal
Modelo y normalización	ER, informe TP1, schema.sql y dependencias funcionales
Integridad y lógica	ENUM, PK, FK, CHECK, UNIQUE, triggers y procedimiento
Concurrencia	transacciones.sql y SELECT FOR UPDATE
Rendimiento	EXPLAIN ANALYZE, índices parciales y vista materializada

2. Modelo relacional y normalización

El modelo parte de cinco entidades: categoria, producto, usuario, pedido y detalle_pedido. Categoria-producto, usuario-pedido y pedido-detalle_pedido son relaciones 1:N. Pedido-producto es N:M y se resuelve mediante detalle_pedido, que conserva cantidad, precio histórico y subtotal por línea.

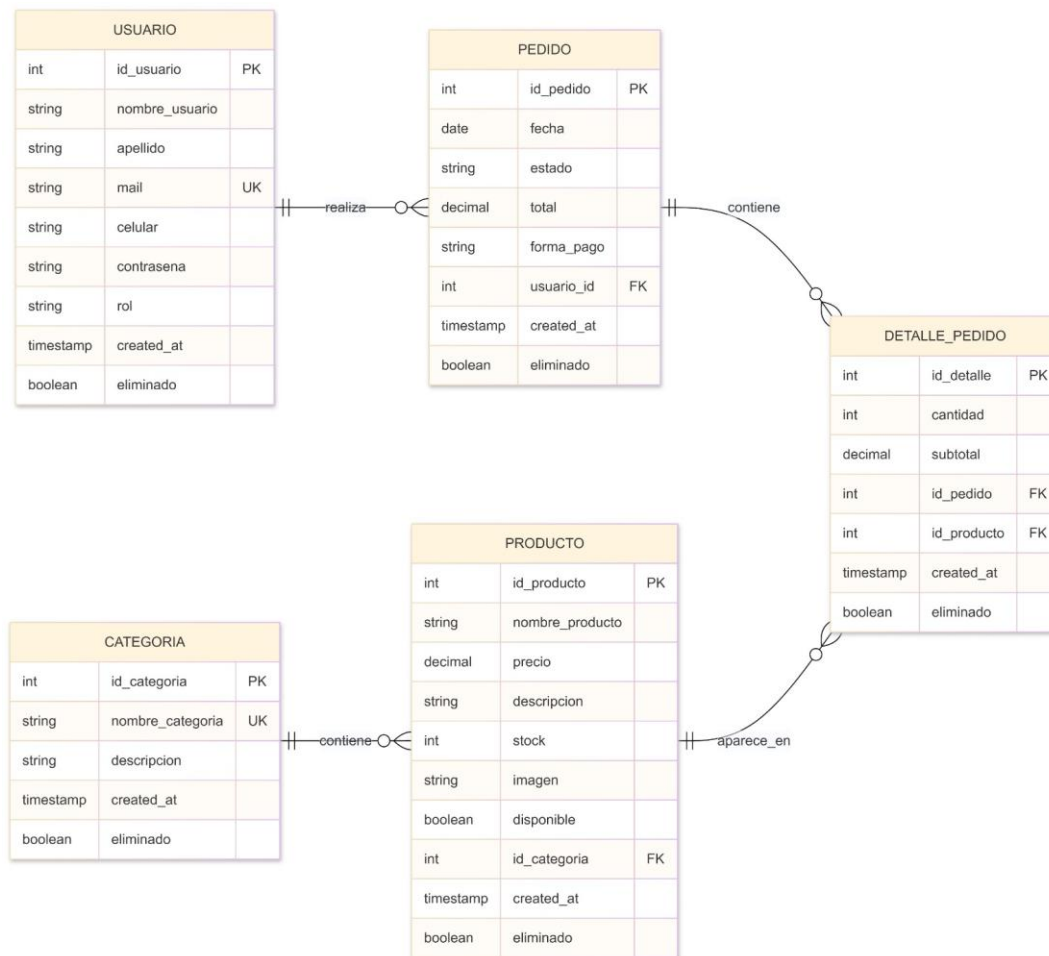


Figura 1. Modelo ER de FoodStore

- 1FN: los ítems del pedido se almacenan como filas de detalle_pedido; no hay grupos repetidos ni atributos multivaluados.
- 2FN: los atributos dependen de la clave de cada relación. En detalle_pedido se preserva la clave natural (id_pedido, id_producto) mediante UNIQUE.
- 3FN/BCNF: categoria y usuario no se duplican en producto o pedido. Los determinantes de las relaciones base son claves candidatas o superclaves.
- precio_unitario fue incorporado en la implementación como snapshot de venta. subtotal y total son desnormalizaciones controladas por triggers para preservar consistencia e histórico.

3. DDL, integridad y objetos programables

sql/schema.sql implementa PK IDENTITY, FK, tipos ENUM, CHECK para precios, stock, cantidades y totales, y UNIQUE para mail, categoría y la combinación pedido-producto. Todas las entidades usan soft delete mediante eliminado = FALSE.

Objeto	Aplicación en FoodStore
Vistas	Vistas vigentes, tres vistas TP5 y una vista materializada para reportes.

Objeto	Aplicación en FoodStore
Funciones y triggers	Cálculo de subtotal y total; validación de transiciones y bajas lógicas.
Procedimiento	sp_crear_pedido valida usuario y stock, bloquea filas con FOR UPDATE e inserta el pedido de forma atómica.
Reglas	Estados finales inmutables y prohibición de baja lógica de categoría con productos vigentes.

4. Consultas, transacciones y pruebas

sql/queries.sql y los archivos de los TPs incluyen JOIN, agregaciones, subconsultas, GROUP BY, HAVING y funciones de ventana. Por ejemplo, carga_masiva_tp3.sql verifica pedidos con GROUP BY id_pedido HAVING count(*) <> 2 y controla mails duplicados con HAVING count(*) > 1.

Las restricciones se probaron con 14 casos dentro de BEGIN/ROLLBACK. Las equivalencias de consultas y vistas se verificaron con EXCEPT en ambas direcciones, obteniendo 0 filas.

Escenario	Resultado validado
Atomicidad	sp_crear_pedido aborta ante validaciones inválidas sin dejar pedidos parciales.
COMMIT y ROLLBACK	Los cambios de stock persisten o se revierten según el desenlace de la transacción.
Aislamiento	READ COMMITTED y SERIALIZABLE se analizaron con dos sesiones concurrentes.
Sobreventa	SELECT FOR UPDATE bloquea la fila de producto y evita descontar más stock del disponible.

5. Optimización de consultas

La estrategia aplicada fue medir primero con EXPLAIN ANALYZE, proponer una alternativa y conservarla solo cuando mejoró el Execution Time. Las propuestas más lentas o poco selectivas se descartaron.

Consulta	Antes	Después	Resultado
Detalle por producto TP3	406,794 ms	16,355 ms	Índice aceptado (~25x)
Consulta común TP3/TP4	93,394 ms	10,855 ms	Índice aceptado (~8,6x)

Consulta	Antes	Después	Resultado
Pedidos cancelados TP5	210,677 ms	2,822 ms	Índice parcial (~74,7x)
Productos no disponibles TP5	31,581 ms	2,767 ms	Índice parcial (~11,4x)
Cantidad excepcional TP5	182,874 ms	1,498 ms	Índice parcial (~122x)
Facturación materializada	706,310 ms	0,284 ms	Vista materializada (~2487x)

También se descartaron índices que empeoraron las consultas de TP3 y un índice simple sobre pedido(eliminado), por su baja cardinalidad. Las reescrituras analíticas del TP4 tampoco se conservaron porque sus tiempos reales aumentaron.

6. Vistas y vista materializada

- v_tp5_productos_categoria muestra productos y categorías vigentes.
- v_tp5_pedidos_usuario expone datos públicos del usuario, sin contraseña, celular ni rol.
- v_tp5_detalle_pedido_productos conserva el nombre del producto histórico aunque el producto esté eliminado lógicamente.
- mv_tp5_facturacion_categoria_mes precalcula facturación por categoría y mes. Usa índice único para habilitar REFRESH MATERIALIZED VIEW CONCURRENTLY.

La vista materializada se refresca por lote o diariamente para reportes. No se utiliza para stock ni pedidos activos porque es una instantánea que puede quedar desactualizada entre refreshes.

7. Uso de inteligencia artificial y conclusión

Kiro se utilizó para especificaciones; OpenCode para propuestas y generación de archivos; Codex para guía, revisión y coordinación. Todas las decisiones aceptadas o descartadas se validaron manualmente en DBeaver. Las DUIA específicas y duia_tpi_parcial.md conservan el detalle de prompts, ajustes y resultados.

La primera entrega parcial deja un proyecto verificable: modelo normalizado, integridad y reglas de negocio en el motor, concurrencia controlada, soft delete coherente y optimización basada en mediciones reales. El repositorio contiene los scripts, pruebas, informes y evidencia de diseño necesarios para su revisión.

Repositorio: https://github.com/francosampieri/TP2_BDD2_FoodStore.git